JAVA FILE HANDLING

Complete Revision Guide • Byte Streams

Scope: The complete byte-stream section: stream fundamentals, hierarchy, file streams, buffered streams, abstract stream classes, data streams, PrintStream, practical programs, comparisons, pitfalls and interview revision.

Learning objective: Understand not only which class to use, but also how stream objects wrap one another, how bytes move, and how to read back data correctly.

---

1. Introduction to File Handling

File: A named collection of data stored on a storage device. Examples include text files, PDFs, images, audio files and videos.

File handling: The process of creating, reading, writing, appending, copying and managing files from a program.

Ordinary variables hold values while a program runs. A file provides persistent storage: its contents can remain available after the program terminates, provided the file is not deleted or overwritten.

Input and output

Operation	Meaning	Example
Input	Data moves from a source into the Java program	Reading a file
Output	Data moves from the Java program to a destination	Writing to a file

Stream: A stream is an abstraction representing a flow or sequence of data from a source to a destination. Java I/O streams provide a common way to read and write data.

Two major stream families

Family	Base classes	Typical use
Byte streams	InputStream and OutputStream	Raw bytes; binary files such as images, audio and PDFs
Character streams	Reader and Writer	Text, with character encoding/decoding handled by reader/writer implementations

Important precision: A byte is 8 bits. Java `char` is a UTF-16 code unit, not always a complete Unicode character. Byte streams transfer bytes; character streams convert between bytes and characters using a charset.

> **Can FileInputStream read a text file?** Yes. Text files are stored as bytes. FileInputStream reads those bytes but does not decode them into text. Use a Reader or an explicit Charset decoder to interpret encoded text.

2. Byte Stream Architecture and Inheritance

The central abstract classes are InputStream for byte input and OutputStream for byte output. Concrete classes implement or extend these abstractions.

Input hierarchy (simplified)	Output hierarchy (simplified)
Object → InputStream (abstract)	Object → OutputStream (abstract)
InputStream → FileInputStream	OutputStream → FileOutputStream
InputStream → FilterInputStream → BufferedInputStream	OutputStream → FilterOutputStream → BufferedOutputStream
InputStream → FilterInputStream → DataInputStream	OutputStream → FilterOutputStream → DataOutputStream

Input hierarchy (simplified)	Output hierarchy (simplified)
	OutputStream → FilterOutputStream → PrintStream

Wrapper/decorator idea: BufferedInputStream, DataInputStream, BufferedOutputStream, DataOutputStream and PrintStream can wrap another stream. The outer stream adds behaviour while delegating the underlying I/O to the wrapped stream.

Example: `new DataInputStream(new BufferedInputStream(new FileInputStream("student.dat")))`. The file stream accesses the file, the buffered stream reduces frequent underlying reads, and the data stream exposes methods such as readInt() and readUTF().

3. FileInputStream

Formal definition: FileInputStream is a byte input stream used to read raw bytes from a file.

Constructors

Constructor	Purpose
FileInputStream(String name)	Opens the file identified by a path string.
FileInputStream(File file)	Opens the specified File object.
FileInputStream(FileDescriptor fd)	Creates a stream using an existing file descriptor.

Important methods

Method	What it does
read()	Reads one byte; returns 0–255, or -1 at end of stream.
read(byte[] b)	Reads bytes into an array; returns number read, or -1 at EOF.
read(byte[] b, int off, int len)	Reads up to len bytes into the specified array range.
available()	Estimates bytes readable without blocking; not a reliable total-file-size method.
skip(long n)	Attempts to skip bytes.
getChannel() / getFD()	Returns the associated FileChannel / FileDescriptor.
close()	Closes the stream and releases associated resources.

Program: read a file one byte at a time

```
import java.io.FileInputStream;
import java.io.IOException;

public class ReadFile {
    public static void main(String[] args) {
        try (FileInputStream fis =
                 new FileInputStream("input.txt")) {

            int data;
            while ((data = fis.read()) != -1) {
                System.out.print((char) data);
            }

        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Example input.txt: Hello Java (followed by a newline) File Handling.

Output: The file's contents are printed. The direct byte-to-char cast is only a simple demonstration for ASCII-compatible content; it is not a correct general UTF-8 decoder.

Why is read() stored in an int?

The method must represent every possible unsigned byte value (0 through 255) and a separate end-of-stream marker (-1). A byte variable cannot represent all these values plus a distinct EOF marker.

Reading a block

```
byte[] buffer = new byte[4096];
int count;

try (FileInputStream in = new FileInputStream("input.bin")) {
    while ((count = in.read(buffer)) != -1) {
        // Process only buffer[0] through buffer[count - 1]
    }
}
```

4. FileOutputStream

Formal definition: FileOutputStream is a byte output stream used to write raw bytes to a file.

Constructors

Constructor	Purpose
FileOutputStream(String name)	Creates or opens a file for writing; truncates existing content.
FileOutputStream(String name, boolean append)	If append is true, writes at the end; otherwise truncates.
FileOutputStream(File file)	Uses a File object; truncates existing content.
FileOutputStream(File file, boolean append)	File-object form with append option.
FileOutputStream(FileDescriptor fd)	Uses an existing file descriptor.

Important methods

Method	What it does
write(int b)	Writes the low-order 8 bits of the integer as one byte.
write(byte[] b)	Writes all bytes in the array.
write(byte[] b, int off, int len)	Writes a selected range of bytes.
flush()	Inherited output-stream operation; forces pending output to the underlying destination where applicable.
close()	Closes the stream.

Program: write UTF-8 text bytes

```
import java.io.FileOutputStream;
import java.io.IOException;
import java.nio.charset.StandardCharsets;

public class WriteFile {
    public static void main(String[] args) {
        String message = "Welcome to Java File Handling";

        try (FileOutputStream fos =
                 new FileOutputStream("output.txt")) {

            byte[] data =
                message.getBytes(StandardCharsets.UTF_8);
            fos.write(data);
```

```
            System.out.println("Data written successfully");
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Output: Data written successfully. The file contains: Welcome to Java File Handling.

Append mode

```
try (FileOutputStream fos =
         new FileOutputStream("output.txt", true)) {
    fos.write("\\nAdditional line".getBytes(StandardCharsets.UTF_8));
}
```

The second constructor argument controls append behaviour: true appends; false overwrites/truncates. The default constructors use false.

> **Common mistake:** Forgetting append=true when adding to an existing file. The default FileOutputStream constructor does not preserve old contents.

5. BufferedInputStream

Formal definition: BufferedInputStream is a filtering byte input stream that adds an internal memory buffer around another InputStream. It can reduce the frequency of underlying read operations.

Constructors

Constructor	Purpose
BufferedInputStream(InputStream in)	Wraps an input stream using the default buffer size.
BufferedInputStream(InputStream in, int size)	Wraps an input stream using the specified buffer size.

Important methods

Method	What it does
read(), read(byte[]), read(byte[], off, len)	Reads bytes, using the internal buffer.
available()	Estimates bytes readable without blocking.
skip(long n)	Skips bytes.
mark(int readlimit)	Marks the current position, if supported.
reset()	Returns to the marked position while the mark remains valid.
markSupported()	Returns whether mark/reset are supported; BufferedInputStream supports them.
close()	Closes the buffered stream and its wrapped input stream.

How buffering works

• The BufferedInputStream asks the underlying stream to fill an internal byte array when needed.

• Individual read() calls can then be served from memory rather than each requiring a new underlying read.

• When the buffer is exhausted, it is refilled.

• Buffering reduces call overhead for many small reads; actual speed depends on workload, OS and filesystem caching.

Program: read using BufferedInputStream

```
import java.io.BufferedInputStream;
import java.io.FileInputStream;
```

```
import java.io.IOException;

public class BufferedRead {
    public static void main(String[] args) {
        try (BufferedInputStream bis =
                 new BufferedInputStream(
                     new FileInputStream("input.txt"), 8192)) {

            int data;
            while ((data = bis.read()) != -1) {
                System.out.print((char) data);
            }
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

The output is the same as the FileInputStream example. The difference is that reads are served through an internal buffer. The byte-to-char cast remains unsuitable for arbitrary encoded text.

6. BufferedOutputStream

Formal definition: BufferedOutputStream is a filtering byte output stream that temporarily accumulates bytes in an internal buffer before forwarding them to its underlying OutputStream.

Constructors

Constructor	Purpose
BufferedOutputStream(OutputStream out)	Wraps an output stream using the default buffer size.
BufferedOutputStream(OutputStream out, int size)	Wraps an output stream using a specified buffer size.

Important methods

Method	What it does
write(int b)	Adds one byte to the buffer.
write(byte[] b)	Writes the byte array through the buffer.
write(byte[] b, int off, int len)	Writes a selected range.
flush()	Forwards buffered bytes to the underlying stream.
close()	Flushes pending bytes and closes the stream.

Understanding flush()

Buffered output may remain in memory until the buffer fills, flush() is called, or the stream is closed. flush() forwards pending bytes; it does not close the stream. Closing a BufferedOutputStream flushes it and closes the wrapped stream.

Program: write using BufferedOutputStream

```
import java.io.BufferedOutputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.nio.charset.StandardCharsets;

public class BufferedWrite {
    public static void main(String[] args) {
        try (BufferedOutputStream bos =
                 new BufferedOutputStream(
                     new FileOutputStream("output.txt"))) {

            String message = "Learning Buffered Output Streams";
            bos.write(message.getBytes(StandardCharsets.UTF_8));
            bos.flush();
```

```
            System.out.println("File written successfully");
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

7. File Streams vs Buffered Streams

Feature	FileInputStream / FileOutputStream	BufferedInputStream / BufferedOutputStream
Role	Accesses a file as bytes	Adds buffering around another stream
Buffer	No additional Java-level buffer of its own	Uses an internal buffer
Constructor	Accepts path, File or descriptor	Accepts an InputStream or OutputStream
Use case	Basic file byte I/O	Repeated small reads/writes; often improves efficiency
Relationship	Underlying file stream	Wrapper around another stream

Interview answer: Buffered streams can improve performance by reducing frequent underlying I/O calls. They do not guarantee a speedup for every workload, because operating systems also cache file data and performance depends on the access pattern.

8. Practical Program: Copy a Binary File

This program copies an image, PDF or other binary file without decoding it as text. It uses block-based reads and writes.

```
import java.io.*;

public class FileCopy {
    public static void main(String[] args) {
        try (
            BufferedInputStream bis =
                new BufferedInputStream(
                    new FileInputStream("source.jpg"));

            BufferedOutputStream bos =
                new BufferedOutputStream(
                    new FileOutputStream("destination.jpg"))
        ) {
            byte[] buffer = new byte[8192];
            int bytesRead;

            while ((bytesRead = bis.read(buffer)) != -1) {
                bos.write(buffer, 0, bytesRead);
            }

            System.out.println("File copied successfully!");
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Why write(buffer, 0, bytesRead)? The final read may fill only part of the array. Writing the entire array could copy unused or stale bytes. The offset-and-length overload writes exactly the number of bytes read.

Try-with-resources: Both streams are closed automatically. Closing the outer buffered streams also closes their wrapped file streams.

9. InputStream

Formal definition: InputStream is the abstract superclass of classes representing byte input streams. It defines common operations for reading bytes from sources such as files, memory and network connections.

You cannot instantiate an abstract class directly. A reference of type InputStream can refer to an instance of a concrete subclass.

```
InputStream input = new FileInputStream("input.txt");
int value;
while ((value = input.read()) != -1) {
    // Process the byte
}
input.close();
```

The declared reference type is InputStream; the actual object is FileInputStream. This is polymorphism and allows code to depend on a general abstraction.

Method	Description
read()	Reads one byte or returns -1 at EOF.
read(byte[])	Reads into an array.
read(byte[], off, len)	Reads into a selected range.
skip(long)	Attempts to skip bytes.
available()	Estimates bytes readable without blocking.
mark(int), reset(), markSupported()	Mark/reset operations where supported.
close()	Closes the stream.

Important: available() is not a general way to find the total size of a stream or to decide that all data has arrived.

10. OutputStream

Formal definition: OutputStream is the abstract superclass of classes representing byte output streams. It defines common operations for writing bytes to destinations.

```
OutputStream output = new FileOutputStream("output.bin");
byte[] data = {65, 66, 67};
output.write(data);
output.flush();
output.close();
```

Method	Description
write(int)	Writes the low-order byte.
write(byte[])	Writes all bytes in an array.
write(byte[], off, len)	Writes a selected range.
flush()	Forces pending output to the destination where applicable.
close()	Closes the stream.

FileOutputStream is a concrete OutputStream. BufferedOutputStream and other output wrappers can also be assigned to an OutputStream reference.

11. DataInputStream

Formal definition: DataInputStream is a filter input stream that lets an application read Java primitive data types and strings from an underlying input stream in a portable binary format.

Constructor

`DataInputStream(InputStream in)` — wraps the supplied InputStream.

Important methods

Method	Reads
readBoolean()	boolean
readByte()	byte
readUnsignedByte()	unsigned byte value as int
readShort() / readUnsignedShort()	short / unsigned short as int
readChar()	char
readInt()	int
readLong()	long
readFloat()	float
readDouble()	double
readUTF()	Modified UTF-8 string
readFully(byte[])	Fills the requested byte array or throws EOFException.
skipBytes(int)	Attempts to skip bytes.

DataInputStream reads primitive values in the format defined by DataOutputStream. It does not automatically discover field names or record boundaries.

12. DataOutputStream

Formal definition: DataOutputStream is a filter output stream that writes Java primitive data types and strings in a portable binary format to an underlying OutputStream.

Constructor

`DataOutputStream(OutputStream out)` — wraps the supplied OutputStream.

Important methods

Method	Writes
writeBoolean(boolean)	boolean
writeByte(int)	low-order byte
writeShort(int)	short
writeChar(int)	char
writeInt(int)	int
writeLong(long)	long
writeFloat(float)	float
writeDouble(double)	double
writeUTF(String)	Modified UTF-8 string with length prefix
write(byte[]) / write(byte[], off, len)	Byte arrays or selected ranges
flush() / close()	Flushes pending output / closes stream
size()	Returns the number of bytes written so far (as an int)

Critical rule: Read values in exactly the same order and with matching read methods as they were written. The file does not contain automatic field names or a schema.

Program: write student data

```
import java.io.*;

public class DataOutputDemo {
    public static void main(String[] args) {
        try (DataOutputStream dos =
                 new DataOutputStream(
                     new FileOutputStream("student.dat"))) {

            dos.writeInt(101);
            dos.writeUTF("Jaideep");
            dos.writeDouble(8.8);
            dos.writeBoolean(true);
            dos.writeChar('A');

            System.out.println("Student data saved");
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Program: read the same student data

```
import java.io.*;

public class DataInputDemo {
    public static void main(String[] args) {
        try (DataInputStream dis =
                 new DataInputStream(
                     new FileInputStream("student.dat"))) {

            int id = dis.readInt();
            String name = dis.readUTF();
            double cgpa = dis.readDouble();
            boolean passed = dis.readBoolean();
            char grade = dis.readChar();

            System.out.println("ID: " + id);
            System.out.println("Name: " + name);
            System.out.println("CGPA: " + cgpa);
            System.out.println("Passed: " + passed);
            System.out.println("Grade: " + grade);
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Expected output: ID: 101; Name: Jaideep; CGPA: 8.8; Passed: true; Grade: A.

writeUTF/readUTF note: These methods use Java's modified UTF-8 format, which is not identical to standard UTF-8. Use the matching DataInputStream/DataOutputStream methods for this format.

Buffered data streams together

```
DataOutputStream dos = new DataOutputStream(
    new BufferedOutputStream(
        new FileOutputStream("student.dat")));

DataInputStream dis = new DataInputStream(
    new BufferedInputStream(
        new FileInputStream("student.dat")));
```

The outer data stream provides primitive methods. The inner buffered stream provides buffering. The file stream accesses the file.

13. PrintStream

Formal definition: PrintStream is an output stream that provides convenient methods for printing representations of values and objects. It can write formatted textual output to an underlying output destination.

Common constructors

Constructor	Purpose
PrintStream(String fileName)	Creates a print stream for a named file.
PrintStream(File file)	Creates a print stream for a File.
PrintStream(OutputStream out)	Wraps an output stream.
PrintStream(OutputStream out, boolean autoFlush)	Configures automatic flushing.
PrintStream(OutputStream out, boolean autoFlush, Charset charset)	Configures automatic flushing and character encoding (modern Java).

Important methods

Method	Description
print(value)	Prints without adding a line terminator.
println(value)	Prints followed by a line terminator.
printf(format, args...)	Prints formatted output.
format(format, args...)	Formatted output; similar to printf.
append(...)	Appends a character or character sequence.
write(...)	Writes bytes.
flush() / close()	Flushes / closes the stream.
checkError()	Flushes if needed and checks the stream's error state.

Program: write formatted student details

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class PrintStreamDemo {
    public static void main(String[] args) {
        try (PrintStream ps =
                 new PrintStream("student.txt",
                                 StandardCharsets.UTF_8)) {

            ps.println("Student Details");
            ps.println("----------------");
            ps.println("Name: Jaideep");
            ps.println("ID: 101");
            ps.printf("CGPA: %.2f%n", 8.8);
            ps.println("Passed: true");

            if (ps.checkError()) {
                System.out.println("An output error occurred");
            } else {
                System.out.println("Data written successfully");
            }
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

File content: Student Details; separator line; Name: Jaideep; ID: 101; CGPA: 8.80; Passed: true.

System.out explained

`System.out` is a static field whose type is PrintStream. Therefore, `System.out.println("Hello")` calls println() on a PrintStream connected to standard output. System.err is another PrintStream, normally connected to standard error.

Important limitation: PrintStream generally suppresses IOExceptions during printing and records an error state. Use checkError() when you need to inspect that state.

14. Complete Byte Stream Comparison

Class	Main purpose	Representative methods
InputStream	Abstract byte input	read(), skip(), available()
OutputStream	Abstract byte output	write(), flush(), close()
FileInputStream	Read file bytes	read()
FileOutputStream	Write file bytes	write()
BufferedInputStream	Buffered byte input	read(), mark(), reset()
BufferedOutputStream	Buffered byte output	write(), flush()
DataInputStream	Read primitive data	readInt(), readDouble(), readUTF()
DataOutputStream	Write primitive data	writeInt(), writeDouble(), writeUTF()
PrintStream	Convenient textual/formatted output	print(), println(), printf()

Choose the class by requirement

Requirement	Suitable class
Read an image or PDF as bytes	FileInputStream
Write binary bytes to a file	FileOutputStream
Repeated small reads from a file	BufferedInputStream
Repeated small writes to a file	BufferedOutputStream
Read primitive values from a binary file	DataInputStream
Write primitive values to a binary file	DataOutputStream
Write convenient formatted text to a file	PrintStream
Write code against a general byte-input abstraction	InputStream
Write code against a general byte-output abstraction	OutputStream

15. Important Concepts and Common Mistakes

• **EOF:** read() returns -1 when the end of the stream is reached. Store the result in int.

• **Append:** FileOutputStream(path, true) appends. The ordinary constructor truncates existing content.

• **Buffering:** Buffered streams wrap another stream; they do not replace the underlying source or destination.

• **Flush:** flush() pushes pending output to the underlying stream. It does not close the stream.

• **Close:** close() releases the resource. Closing an outer wrapper closes its wrapped stream.

• **Partial block reads:** Always use the actual count returned by read(buffer), especially for the final block.

• **Binary vs text:** Do not convert arbitrary image/PDF bytes to characters. Copy raw bytes.

• **Encoding:** String.getBytes() without a Charset uses the default charset. Prefer StandardCharsets.UTF_8 when UTF-8 is intended.

• **Data stream order:** Read fields in the same order and type used when writing them.

• **Data stream format:** DataOutputStream is not Java object serialization and does not store field names.

• **available():** It is not a reliable total file size or end-of-stream test.

• **PrintStream:** Printing methods generally do not throw IOExceptions; checkError() can inspect the error state.

16. Try-with-resources and Exception Handling

File operations can fail, for example because a file is missing, permission is denied, or a device error occurs. Many stream operations declare IOException. FileInputStream construction can also throw FileNotFoundException.

```
try (FileInputStream in = new FileInputStream("input.bin")) {
    int value;
    while ((value = in.read()) != -1) {
        // process byte
    }
} catch (IOException e) {
    System.out.println("I/O error: " + e.getMessage());
}
```

Try-with-resources automatically closes every AutoCloseable resource declared in its resource header, including when an exception occurs. If multiple resources are declared, they are closed in reverse order of creation.

17. Interview Questions and Answers

1. What is a byte stream?
A stream that reads or writes data as 8-bit bytes. It is suitable for raw binary data.

2. What is the parent class for byte input?
InputStream, an abstract class.

3. What is the parent class for byte output?
OutputStream, an abstract class.

4. Why does read() return int instead of byte?
To represent all byte values from 0 to 255 and a separate EOF value of -1.

5. Difference between FileInputStream and BufferedInputStream?
FileInputStream reads bytes from a file. BufferedInputStream wraps an InputStream and adds an internal buffer.

6. Why use BufferedOutputStream?
It accumulates bytes and reduces frequent underlying write operations.

7. Does flush() close a stream?
No. It forwards pending output; the stream remains open.

8. What does append=true do?
It writes at the end of the existing file rather than truncating it.

9. Why use DataInputStream?
It provides methods to read primitive types and modified UTF strings from a compatible binary data stream.

10. Why must data be read in the same order it was written?
The data stream does not store field names or a schema; the reader must know the binary layout.

11. Difference between DataOutputStream and PrintStream?
DataOutputStream writes primitive values in a binary format. PrintStream provides convenient textual and formatted printing.

12. What is System.out?
A static PrintStream field used for standard output.

13. Can byte streams read text?
Yes. They read the underlying bytes, but do not decode them into characters automatically.

14. What is try-with-resources?
A try statement that automatically closes declared AutoCloseable resources.

15. Why use write(buffer, 0, bytesRead)?
It writes only the valid bytes read, including when the final block is smaller than the buffer.

18. Practice Programs

• Create a file and write your name, college and branch using FileOutputStream.

• Read the file using FileInputStream and display its bytes safely for known ASCII content.

• Append a new line using FileOutputStream with append=true.

• Read a file using BufferedInputStream with a custom buffer size of 4096.

• Copy a PDF or image using BufferedInputStream and BufferedOutputStream.

• Copy a file and calculate the total number of bytes copied.

• Write an integer, a UTF string, a double and a boolean using DataOutputStream; read them back using DataInputStream.

• Store five student records by writing the record count first, then each field in a fixed order.

• Write a formatted student report using PrintStream.

Suggested byte-counting copy exercise

```
long totalBytes = 0;
byte[] buffer = new byte[8192];
int count;

while ((count = bis.read(buffer)) != -1) {
    bos.write(buffer, 0, count);
    totalBytes += count;
}
System.out.println("Bytes copied: " + totalBytes);
```

19. Quick Revision Sheet

Remember	Key point
Byte unit	8 bits
Input parent	InputStream
Output parent	OutputStream
File read	FileInputStream
File write	FileOutputStream
Buffered read	BufferedInputStream
Buffered write	BufferedOutputStream
Read primitive	DataInputStream
Write primitive	DataOutputStream
Formatted printing	PrintStream
End of stream	read() returns -1
Append	new FileOutputStream(path, true)

Remember	Key point
Flush	Push pending output; does not close
Close	Release stream resource
Safe block copy	write(buffer, 0, bytesRead)
Text encoding	Use an explicit Charset when converting text and bytes

Self-check: You should be able to explain the difference between direct and buffered streams, write and read a primitive-data file, copy a binary file, and explain why the order of DataInputStream reads must match the DataOutputStream writes.